

Validation Study Inventory

This report maps the repo's validation studies to the files they create/use, and documents the exact data ingestion → processing flow for the stock market engine and indicator validation.

1. Main Validation Study Artifacts

A. Data Lab / CSV validation

- `PythonDataService/app/services/validation\_service.py`
 - Generates markdown validation reports comparing pandas-ta output vs TradingView CSV e
 - Contains the full row-by-row comparison logic and divergence grading.
- `PythonDataService/app/routers/validation\_study.py`
 - Implements the Validation Study API.
 - Exposes `/run`, `/export-csv`, and `/report` endpoints.
- `docs/data-lab-roadmap.md`
 - Describes Data Lab goals, current state, and how the validation report should be rend
- `docs/csv-data-pipeline-plan.md`
 - Defines the step-by-step data cleanup and validation plan for minute OHLCV data.
- `validation\_report\_SPY\_15m.pdf`
 - Example/derived report artifact used for SPY 15-minute validation.
- `PythonDataService/study\_report.txt`
 - Text report from a data or research validation study.
- `PythonDataService/study\_full\_output.txt`
 - Full raw output capture for a validation study run.
- `PythonDataService/study\_output.txt`
 - Condensed summary output for a validation study.

B. Engine parity and LEAN validation

- `docs/lean-engine-phase1-verification-report.md`
 - Phase 1 verification report for the in-process backtest engine.
- `docs/spy-lean-output-report.md`
 - LEAN output reference and calculation guide for SPY EMA crossover.
- `PythonDataService/app/engine/tests/test\_spy\_validation.py`
 - Bit-exact trade parity test against LEAN reference trade log.
- `PythonDataService/app/engine/tests/test\_spy\_next\_bar\_open\_validation.py`
 - NEXT_BAR_OPEN fill-mode parity validation.
- `PythonDataService/app/engine/tests/test\_sma\_crossover\_parity.py`
 - Cross-engine parity for SMA crossover strategy.
- `PythonDataService/app/engine/tests/test\_rsi\_mean\_reversion\_parity.py`
 - RSI mean-reversion parity test.
- `PythonDataService/app/engine/tests/test\_lean\_daily\_reader\_parity.py`
 - Parity test for the LEAN daily data reader.

C. Indicator and portfolio validation UI and services

- `Frontend/src/app/components/indicator-validation/\*`
 - Frontend components for indicator validation reports.
- `Frontend/src/app/components/strategy-lab-validation/\*`
 - Validation UI for strategy lab results.
- `Frontend/src/app/components/portfolio/validation/\*`
 - Portfolio validation dashboard UI.
- `Backend/Services/Implementation/PortfolioValidationService.cs`
 - Backend implementation of portfolio validation rules.

- `Backend/Services/Interfaces/IPortfolioValidationService.cs`
 - Portfolio validation interface contract.
- `Backend/Models/Portfolio/ValidationResult.cs`
 - Validation result model definitions.
- `docs/portfolio-validation-plan.md`
 - Validation plan for portfolio rules and accounting integrity.

2. Data Ingestion → Processing → Validation Flow

Step 1: Data Ingestion

1. `PythonDataService/app/services/polygon\_client.py`
 - Fetches raw minute OHLCV aggregates from Polygon.io.
 - Supports chunked requests and pagination for long date ranges.
2. `PythonDataService/app/services/dataset\_service.py` - Receives raw minute bars.
 - Normalizes field names and timestamps.
 - Tags sessions and applies NYSE RTH filtering.

Step 2: Data Quality and Cleanup

1. `PythonDataService/app/services/data\_quality\_service.py`
 - Cleans minute bars through a 7-step pipeline.
 - Steps may include:
 - RTH session filter
 - fractional volume correction
 - zero-volume and flat-bar removal
 - VWAP recomputation
 - OHLC rule enforcement
 - duplicate/drop handling
 - gap detection
2. Known issue handling
 - Late settlement bars around 07:00 ET are identified as contamination.
 - Data Lab plan explicitly calls this out as the #1 source of TradingView divergence.

Step 3: Indicator Calculation

1. `PythonDataService/app/services/dataset\_service.py`
 - `calculate\_dynamic\_indicators()` invokes `pandas\_ta`.
 - It auto-detects required inputs (open/high/low/close/volume) per indicator.
 - `rename\_to\_indicator\_table\_columns()` normalizes pandas-ta names to contract names
2. Indicator outputs include:
 - EMA,
 - SMA,
 - MACD,
 - Bollinger Bands,
 - RSI,
 - ADX,
 - Supertrend,
 - VWAP,
 - additional derived fields.

Step 4: Strategy Execution and Backtest Validation

1. `PythonDataService/app/engine/engine.py`
 - Runs the backtest engine over cleaned bars.
 - Uses `BacktestEngine`, strategy classes, and fill models.

2. Strategy validation tests compare against reference outputs:

- `SpyEmaCrossoverAlgorithm` vs LEAN trade log.
- `SMA crossover` vs legacy pandas-ta strategy.
- `RSI mean reversion` parity.
- `LeanDailyDataReader` parity.

Step 5: Report Generation and Comparison

1. Validation report generation

- `PythonDataService/app/services/validation\_service.py` compares CSV exports and prod
- It calculates row alignment, exact/close/acceptable/divergent counts, and top diverg

2. Validation Study API

- `PythonDataService/app/routers/validation\_study.py` wraps backtest results and compa
- Outputs include chart bars, indicator data, match stats, and trade comparison tables

3. Frontend visualization

- Angular components in `Frontend/src/app/components/\*validation\*` render strategy, in

3. Files Produced by Each Validation Workflow

Data Lab / CSV Validation

- `validation\_report\_SPY\_15m.pdf` – example formatted report artifact.
- `PythonDataService/study\_report.txt` – summary of study findings.
- `PythonDataService/study\_full\_output.txt` – full logged output from a study run.
- `PythonDataService/study\_output.txt` – condensed summary output.
- Downloaded/exported CSV files from `/export-csv` API (bars, trades, comparison, columns

Engine Parity Validation

- `PythonDataService/app/engine/tests/fixtures/spy\_lean\_trades.csv` – reference trade log
- LEAN output files referenced by `docs/spy-lean-output-report.md` and `docs/lean-engine-
- Strategy logs and report prints from parity tests.

Portfolio Validation

- Backend portfolio validation results exposed as `ValidationResult` objects.
- Frontend validation report components that visualize integrity checks.

4. Recommended Visualization Layout

Use the following report structure in the UI/docs:

1. **Validation Study Summary** - Study name - Ticker / timeframe - Source
data path - Execution timestamp - Result status (pass/fail / grade)

2. **Data Ingestion Path** - Raw source: `Polygon.io` →
`PolygonClientService` - Normalization: `dataset\_service.py` - Quality
cleanup: `data\_quality\_service.py` - Indicator calc:
`calculate\_dynamic\_indicators()`

3. **Validation Output Files** - `study\_report.txt` -
`study\_full\_output.txt` - `study\_output.txt` -
`validation\_report\_SPY\_15m.pdf`

4. **Exact Processing Steps** - Fetch minute bars - Normalize timestamps /
session tag - Filter RTH / drop extended hours as needed - Fix fractional
volume and malformed bars - Forward-fill or gap-fill missing minutes -

Calculate indicators with pandas-ta - Run backtest / strategy - Compare to reference trades or TradingView CSV - Generate markdown + visual report

5. **Divergence Hotspots** - Indicators with highest divergence - Known root causes (07:00 ET contamination, feed mismatch, session mismatch) - Top rows/dates by error magnitude

6. **Validation Contracts** - Bit-exact parity for SPY EMA crossover - Column contract mapping for indicator names - Portfolio integrity invariants from `PortfolioValidationService`

6. pandas-ta → TradingView Column Mapping

This mapping reflects the normalization performed by `PythonDataService/app/services/data

pandas-ta raw field	Normalized field	TradingView export / validation contract
----- ----- -----		
`ema_length<N>`	`ema_<N>`	`ema_5`, `ema_10`, `ema_20`, etc.
`rsi`	`rsi`	`rsi`
`macd`	`macd`	`macd`
`macds`	`macd_signal`	`macd_signal`
`macdh`	`macd_histogram`	`macd_histogram`
`bbl`	`bb_lower`	`bb_lower`
`bbm`	`bb_basis`	`bb_basis`
`bbu`	`bb_upper`	`bb_upper`
`supertl`	`supertrend_up`	`supertrend_up`
`superts`	`supertrend_down`	`supertrend_down`
`adx`	`adx`	`adx`

Notes: - `dmp` / `dmn` from `adx` are intentionally dropped by the contract.
- The indicator-table contract only keeps columns that match the normalized schema; other raw pandas-ta outputs are dropped.

7. Final Implementation Notes

- Dashboard conversion: yes, this report should become a frontend dashboard with clickable
- System-generated report writer: no, do not add one.
- Indicator field mapping: included above.

Implementation result

- Created `docs/validation-study-inventory.md` for inspection.
- Generated `docs/validation-study-inventory.pdf` for readable delivery.